



EC-200 Data Structures

LAB MANUAL # 02

Course Instructor: Lecturer Anum Abdul Salam

Lab Engineer: Lab Engineer Ayesha Batool

Student Name: _____

Degree/ Syndicate: _____

	Trait	Obtained Marks	Maximum Marks
R1	Application Functionality 20%		20
R2	Specification & Data structure implementation 30%		30
R3	Reusability 10%		10
R4	Input Validation 10%		10
R5	Efficiency 20%		20
R6	Delivery 10%		10
R7	Plagiarism above 80%		1

Total		10
-------	--	----

Total Marks = *Obtained Marks* ($\sum_1^6 R_i * R_7$)

Lab #02

Introduction to pointers and dynamic memory

Lab Objective:

To practice pointers and dynamic memory applications

Lab Description:

A **pointer** is a variable whose value is the address of another variable. Like any variable or constant, you must declare a pointer before you can work with it. **type *var-name;** There are few important operations performed when using pointers are

1. Define a pointer variable.
2. Assign the address of a variable to a pointer.
3. Access the value at the address available in the pointer variable.

Defining a Pointer Variable	Pointer Variables Assignment:	To access num using iptr and indirection operator *
int *iptr; // iptr can hold the address of an int	int num=25; iptr = #	cout << iptr; // prints 0x4a00 cout << *iptr; // prints 25



Figure 1.1 Dereferencing pointers to access memory location

Value of memory location towards which pointer is pointing can be changed using pointers
*ptr_nam=val; For example *iptr=8;

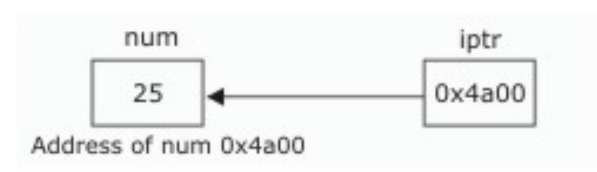


Figure 1.2: Changing the content of location using pointers

It is always a good practice to assign the pointer NULL to a pointer variable in case you do not have exact address to be assigned.

Example

```
void main () {  
int *ptr=NULL;  
if (ptr) { // succeeds if p is not null}  
if (!ptr) { // succeeds if p is null}  
}
```

If all unused pointers are given the null value you can avoid the accidental misuse of an uninitialized pointer. Many times, uninitialized variables hold some junk values and it becomes difficult to debug the program.

Pointer Rules Summary

1. A pointer stores a reference to its pointee. The pointee, in turn, stores something useful.
2. The dereference operation on a pointer accesses its pointee. A pointer may only be dereferenced after it has been assigned to refer to a pointee. Most pointer bugs involve violating this one rule.
3. Allocating a pointer does not automatically assign it to refer to a pointee. Assigning the pointer to refer to a specific pointee is a separate operation which is easy to forget.
4. Assignment between two pointers makes them refer to the same pointee which introduces sharing.

Static Memory Allocation:

In the static memory allocation, the amount of memory to be allocated is predicted and pre known Memory is allocated during the compilation itself. All the declared variables declared normally, are allocated memory statically.

Example

<pre>void main { int a=9; cout<<sqr(a);</pre>	<pre>nt sqr(int val) { return val*val; }</pre>
---	--

}	
---	--

Dynamic Memory Allocation:

Variables declared and used locally inside a function definition are destroyed, when declared variables within a function scope are needed to be used by other functions without creating an overhead by copying these variables via function's return value.

1. Allocate memory when required
2. Use it by initializing to some value
3. Deallocate the memory when no more required

Memory Allocation	Memory Initialization	Memory deallocation
<pre> ptrtype* ptrname; pvalue = new ptrtype; void main { int*p; p=new int; } </pre>	<pre> *ptrname=value; void main { int*p; p=new int; *p=9; } </pre>	<pre> delete ptrname; void main { int*p; p=new int; *p=9; delete p; } </pre>

Dangling Pointers:

The delete operator does not delete the pointer it takes the memory being pointed to and returns it to the heap. It does not even change the contents of the pointer. Since the memory being pointed to is no longer available (and may even be given to another application), such a pointer is said to be dangling.

Memory Leaks and Bad pointers

Memory leaks when it is allocated using the new operator but not returned to the heap using the delete operator. When a pointer is dereferenced but not allocated any memory, such pointer is called bad pointer.

Lab Tasks:

1. For each of the following, write a single statement that performs the specified task. Assume that floating-point variables number1 and number2 have been declared and that number1 has been initialized to 7.3.
 - a) Declare the variable fPtr to be a pointer to an object of type double and initialize the pointer to nullptr.
 - b) Assign the address of variable number1 to pointer variable fPtr.

- c) Display the value of the object pointed to by fPtr.
- d) Assign the value of the object pointed to by fPtr to variable number2.
- e) Display the value of number2.
- f) Display the address of number1.
- g) Display the address stored in fPtr. Is the address displayed the same as that of number1?

2. Create a class **IntStaticPointer** containing a pointer to integer as attribute. Each instance of class should be capable of pointing towards integer variable. Provide
 - **default constructor** Initializes pointer to some initial value (NULL)
 - **void setValue(int&val)** Points the pointer to val
 - **intgetValue()** returns the contents of memory to which pointer is pointing
 - **Destructor** that initializes the pointer again with NULL

3. Find errors and correct the following code fragments:

Code Fragment	Error Correction	Code Fragment	Error Correction
<pre>void main { int * ptr, *foo, h; intval=9; ptr=&val; foo=ptr; *foo=10; int z=*ptr; cout<<val<<" "<<z; }</pre>		<pre>void BadPointer() { int* p; *p = 42; }</pre>	
<pre>void main { int *p; intval=9; p=val; cout<<p; }</pre>		<pre>void main() { int*ptr; ptr=new int; cin>>*ptr; ptr=new int; cin>>*ptr; }</pre>	
<pre>void main{ int a = 5, b = 10; int *p1, *p2; p1 = &a; p2 = &b; *p1 = 10; p1 = p2; *p1 = 20; cout<<a<<" "<<b; }</pre>		<pre>void main() { int*ptr; ptr=new int; cin>>*ptr; ptr=NULL; delete ptr; cout<<*ptr; }</pre>	

4. Create a class **IntDynamicPointer** containing a pointer to integer as attribute. Each instance of class should be capable of pointing towards integer variable.

Provide `

- **default constructor** Initializes pointer to some initial value (NULL)
- **void AllocMem()** allocate memory to the pointer using dynamic memory
- **void setValue(intval)** Points the pointer to val
- **intgetValue()** returns the contents of memory to which pointer is pointing
- **Destructor** that deletes the memory and initializes the pointer again with NULL

5. What happens at runtime when the bad pointer is dereferenced?

6. Find the error in each of the following program segments. Assume the following declarations and statements:

```
int *zPtr; // zPtr will reference built-in array z
```

```
void *sPtr = nullptr;
```

```
int number;
```

```
int z[ 5 ] = { 1, 2, 3, 4, 5 };
```

1. ++zPtr;
2. // use pointer to get first value of a built-in array
number = zPtr;
3. // assign built-in array element 2 (the value 3) to number
number = *zPtr[2];
4. // display entire built-in array z
for (size_t i = 0; i <= 5; ++i)
cout << zPtr[i] << endl;
5. // assign the value pointed to by sPtr to number
number = *sPtr;
6. ++z;

7. Introduce int variables x and y and int* pointer variables p and q. Set x to 2, y to 8, p to the address of x, and q to the address of y. Then print the following information:

Sr.	Commands	Output
1	The address of x and the value of x.	
2	The value of p and the value of *p.	
3	The address of y and the value of y.	
4	The value of q and the value of *q.	

5	The address of p (not its contents!).	
6	The address of q (not its contents!).	

Use the Hex function to print all pointer/address values and format the output so it is easy to make comparisons.

8. Providing that the declarations

```
int intArray[]={1,2,3}, *p=intArray;
```

What will be content of intArray and p after executing

- `*p++;`
- `(*p)++;`
- `*p++ ;`
- `(*p)++;`